Name : Tanmay Mahesh Ahuja **Semester :** V **Branch :** AI&DS

Roll No : 03 **Batch :** T11 **Date :** 18-07-2026

Experiment No. 01

Aim : To perform version control operations using Git.

LO Mapped : LO2

LO : Use Git effectively for version control, source code management, and collaborative team development.

Theory :

1. Introduction to Version Control Systems (VCS)

A Version Control System (VCS) is software tracking the evolution of a codebase over time. It records modifications made to source files, allowing developers to review history, undo changes, and collaborate efficiently. In modern software engineering, version control is the baseline for DevOps automation and Agile lifecycles.

Centralized vs. Distributed Version Control

- **Centralized VCS (CVCS):** Relies on a single central server containing all versioned files. Developers check out individual files from this single point. If the server goes offline, collaboration halts, and a server failure risks data loss.
- **Distributed VCS (DVCS) - Git:** Every developer maintains a full copy of the project repository locally, including its complete historical log tree. Collaboration occurs by syncing these local repositories through remote architectures. This provides offline functionality, fast operation execution, and high redundancy.

2. Git Architecture and Code Lifecycles

Git manages files by transitioning them across four core architectural zones:

1. **Working Directory:** The local file system directory where project files are created, viewed, and modified. Changes here are considered *untracked or modified*.
2. **Staging Area (Index):** A technical preparation cache holding modified assets selected to be included in the next snapshot. It acts as an intermediate gatekeeper for organizing clean, logical changes.
3. **Local Repository (.git):** The internal database hidden within the project folder containing all compressed tracking files, commit objects, history logs, and branch pointers.
4. **Remote Repository:** A centralized server host (e.g., GitHub, GitLab) enabling distributed teams to push, pull, and sync their locally validated changes.

3. Core Git Operations and Workflows

To successfully run a version-controlled project, Git operations are split into logical phases reflecting an automated engineering pipeline.

Phase A: Configuration & Initialization

Before tracking code, a developer defines their signature for the log history.

- **Identity Setup (git config):** Establishes the author's name and email globally or locally to maintain audit transparency.
- **Initialization (git init):** Injects a hidden .git folder into an active project folder, converting a standard directory into a live, tracking Git repository.
- **Status Inquiries (git status):** Periodically reviews the current state of files across the working directory and staging zones.

Phase B: The Local Tracking Loop

- **Staging Changes (git add):** Moves targeted file modifications out of the active working space and writes them straight into the staging index cache.
- **Unstaging Files (git rm --cached / git restore --staged):** Pulls files back out of the staging index without altering the physical code inside the local working directory.
- **Renaming & Moving (git mv):** Modifies file system naming rules while automatically recording and staging the path variance in Git.

Phase C: Inspection & Delta Auditing

Visibility is critical in Agile sprints to ensure changes match task descriptions.

- **Log Diagnostics (git log, git log --oneline):** Displays sequential arrays of chronological commits, capturing cryptographic SHA hashes, author footprints, and contextual messages.
- **Difference Analytics (git diff, git diff --staged):** Compares code variations line by line, demonstrating additions and deletions before or after staging them.

Phase D: Agile Branching & Workspace Isolation

Agile workflows isolate user stories and bug fixes into dedicated sandboxes before integrating them into production lines.

- **Branch Allocation (git branch, git branch -d):** Creates or removes independent structural pointers within the commit history graph.
- **Context Switching (git checkout):** Transitions the active working directory space from one tracking branch line to another.
- **Integration (git merge):** Reconciles and combines tracking logs from a specialized feature line directly into a target base branch (e.g., main).

Phase E: Stash Index Manipulation

- **Data Shelving (git stash, git stash pop):** Temporarily saves uncommitted, incomplete feature states onto an internal memory stack. This cleans the working

directory instantly, allowing developers to switch tasks and later re-apply their cached modifications.

4. Deep-Dive: The 7 Mandated DevOps Operations

The following seven core operations dictate how code changes are validated, stored, secured, shared, and recovered across team-driven software environments.

[Local Workspace] ----(Commit)----> [Local Repository] ----(Push)----> [Remote Server]

[Local Workspace] <---(Clone/Pull)---------- [Remote Server]

1. Commit

- **Definition:** A commit acts as an immutable cryptographic snapshot capturing the precise state of staged code at a point in time.
- **Mechanism:** Every commit receives a unique SHA-1 or SHA-256 identifier hash. This hash tracks the directory hierarchy layout, absolute parent histories, author details, and commit messages. In Agile methodology, a commit functions as the primary baseline verifying an isolated unit of work is ready for compilation.

2. Authorisation

- **Definition:** The validation layer proving a developer has administrative clearance to read from or write to a target remote host server.
- **Mechanism:** Git handles permission structures via HTTPS access tokens or secure cryptographic SSH keys. It integrates with system components like the Git Credential Manager to verify compliance parameters before enabling code transfers, establishing a secure audit trail for DevOps deployment pipelines.

3. Push

- **Definition:** The outbound operational process of transmitting local repository commits up to a remote host branch.
- **Mechanism:** Pushing syncs local branch history lines with remote tracking pointers, providing code visibility to the rest of the engineering team and triggering Continuous Integration (CI) automated testing pipelines.

4. Pull

- **Definition:** An inbound operation that fetches external modifications from a remote repository and instantly blends them into the developer's active local branch.
- **Mechanism:** git pull is a composite command. It runs a network fetch (git fetch) to download remote data updates, followed immediately by a merge command (git merge) to integrate those changes into the current working directory.

5. Clone

- **Definition:** The foundational action of deep-copying an existing external remote repository layout onto a local workstation.

- **Mechanism:** Cloning automatically initializes a new local directory, creates a complete download of the tracking tree database, establishes remote server link shortcuts (conventionally mapped as origin), and checks out the default primary working branch.

6. Revert

- **Definition:** A safe, non-destructive undo action that cancels out the changes introduced by an earlier mistake without rewriting history.
- **Mechanism:** git revert evaluates a specific historical commit hash, computes its exact functional inverse, and automatically deploys a **brand new commit** implementing that inverse logic. Because past history is left unaltered, it is the safest method for correcting bugs in shared production environments.

7. Rollback (Reset Mechanics)

- **Definition:** A structural adjustment that moves the current branch pointer backward in time to a previous stable state, effectively removing subsequent commits from the branch timeline.
- **Mechanism:** Executed via reset protocols, rollbacks occur in three states:
    - *Soft Reset:* Moves the history pointer back, but keeps all changes staged in the index.
    - *Mixed Reset:* Moves the pointer back and clears the index, leaving changes unstaged in the local directory.
    - *Hard Reset:* Destructively rewrites the timeline, wiping away all intermediate commits, staged code, and local file adjustments back to the target commit state.

5. Role of Git in Agile and DevOps

- **Continuous Integration / Continuous Deployment (CI/CD):** Triggers automated code compilation, code quality reviews, and testing engines immediately upon receiving repository updates (git push).
- **Agile Sprint Isolation:** Feature branching allows multi-developer teams to work concurrently on user stories without blocking or corrupting production-ready source files.
- **Traceability:** Links individual lines of code changes back to specific sprint tickets and author identifiers, simplifying bug isolation and root-cause analysis.

Implementation :

Phase 1: Environment Initialization

```
PS C:\Users\tanma> Remove-Item -Recurse -Force .\remote-mock.git -ErrorAction SilentlyContinue
PS C:\Users\tanma> Remove-Item -Recurse -Force .\cloned-project -ErrorAction SilentlyContinue
PS C:\Users\tanma> git config --global user.name "Tanmay Ahuja"
PS C:\Users\tanma> git config --global user.email "tanmayahuja@hotmail.com"
PS C:\Users\tanma> git init --bare remote-mock.git
Initialized empty Git repository in C:/Users/tanma/remote-mock.git/
PS C:\Users\tanma> mkdir git-devops-practical


    Directory: C:\Users\tanma


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        14-07-2026     22:13                git-devops-practical


PS C:\Users\tanma> cd git-devops-practical
PS C:\Users\tanma\git-devops-practical> git init
Initialized empty Git repository in C:/Users/tanma/git-devops-practical/.git/
PS C:\Users\tanma\git-devops-practical>
```

Phase 2: File Creation & Core Commits

```
PS C:\Users\tanma\git-devops-practical> @'
>> def calculate_mean(data):
>>     if not data: return 0
>>     return sum(data) / len(data)
>>
>> numbers = [10, 20, 30, 40, 50]
>> print(f"Mean: {calculate_mean(numbers)}")
>> '@ | Out-File -FilePath analytics.py -Encoding utf8
PS C:\Users\tanma\git-devops-practical> git add analytics.py
warning: in the working copy of 'analytics.py', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\tanma\git-devops-practical> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   analytics.py

PS C:\Users\tanma\git-devops-practical> git commit -m "Initial: added mean calculation algorithm"
[master (root-commit) bf259fd] Initial: added mean calculation algorithm
 1 file changed, 6 insertions(+)
 create mode 100644 analytics.py
```

Phase 3: Modifications & The Revert Operation

```
PS C:\Users\tanma\git-devops-practical> @'
>> def calculate_mean(data):
>>     if not data: return 0
>>     return sum(data) / len(data)
>>
>> def calculate_median(data):
>>     if not data: return 0
>>     s = sorted(data)
>>     return s[len(s)//2]
>>
>> numbers = [10, 20, 30, 40, 50]
>> print(f"Mean: {calculate_mean(numbers)}")
>> print(f"Median: {calculate_median(numbers)}")
>> '@ | Out-File -FilePath analytics.py -Encoding utf8
PS C:\Users\tanma\git-devops-practical> git commit -am "Feat: added median functionality to analytics"
warning: in the working copy of 'analytics.py', LF will be replaced by CRLF the next time Git touches it
[master 1bfab81] Feat: added median functionality to analytics
 1 file changed, 6 insertions(+)
PS C:\Users\tanma\git-devops-practical> git revert HEAD --no-edit
[master 9ea0c56] Revert "Feat: added median functionality to analytics"
 Date: Tue Jul 14 22:19:03 2026 +0530
 1 file changed, 6 deletions(-)
PS C:\Users\tanma\git-devops-practical> git log --oneline
9ea0c56 (HEAD -> master) Revert "Feat: added median functionality to analytics"
1bfab81 Feat: added median functionality to analytics
bf259fd Initial: added mean calculation algorithm
PS C:\Users\tanma\git-devops-practical>
```

Phase 4: Agile Branching & The Rollback Operation

```
PS C:\Users\tanma\git-devops-practical> git checkout -b feature-variance
Switched to a new branch 'feature-variance'
PS C:\Users\tanma\git-devops-practical> Add-Content -Path analytics.py -Value "`n# Experimental tracking code block"
PS C:\Users\tanma\git-devops-practical> git commit -am "Feat: add experimental variance draft"
warning: in the working copy of 'analytics.py', LF will be replaced by CRLF the next time Git touches it
[feature-variance ec821c6] Feat: add experimental variance draft
 1 file changed, 2 insertions(+)
PS C:\Users\tanma\git-devops-practical> git reset --hard HEAD~1
HEAD is now at 9ea0c56 Revert "Feat: added median functionality to analytics"
PS C:\Users\tanma\git-devops-practical> git status
On branch feature-variance
nothing to commit, working tree clean
PS C:\Users\tanma\git-devops-practical>
```

Phase 5: Pipeline Integration (Authorization, Push, Pull, Clone)

```
PS C:\Users\tanma\git-devops-practical> git checkout master
Switched to branch 'master'
PS C:\Users\tanma\git-devops-practical> $remotePath = Resolve-Path "../remote-mock.git"
PS C:\Users\tanma\git-devops-practical> git remote add origin $remotePath
PS C:\Users\tanma\git-devops-practical> git remote -v
origin  C:\Users\tanma\remote-mock.git (fetch)
origin  C:\Users\tanma\remote-mock.git (push)
PS C:\Users\tanma\git-devops-practical> git config credential.helper
manager
PS C:\Users\tanma\git-devops-practical> git push -u origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 789 bytes | 131.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
To C:\Users\tanma\remote-mock.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

```
PS C:\Users\tanma\git-devops-practical> git pull origin master
From C:\Users\tanma\remote-mock
 * branch            master     -> FETCH_HEAD
Already up to date.
PS C:\Users\tanma\git-devops-practical> cd ..
PS C:\Users\tanma> git clone ./remote-mock.git cloned-project
Cloning into 'cloned-project'...
done.
PS C:\Users\tanma> cd cloned-project
PS C:\Users\tanma\cloned-project> git log --graph --oneline --all
* 9ea0c56 (HEAD -> master, origin/master, origin/HEAD) Revert "Feat: added median functionality to analytics"
* 1bfab81 Feat: added median functionality to analytics
* bf259fd Initial: added mean calculation algorithm
PS C:\Users\tanma\cloned-project> git log
commit 9ea0c56490975e7a50f0dd99a50ff2edc739a3b2 (HEAD -> master, origin/master, origin/HEAD)
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:19:03 2026 +0530

    Revert "Feat: added median functionality to analytics"

    This reverts commit 1bfab817a8feaf153ea1203122e681846719b854.

commit 1bfab817a8feaf153ea1203122e681846719b854
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:18:45 2026 +0530

    Feat: added median functionality to analytics

commit bf259fde0a649019ce4b0b93de0438d0c1e47653
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:15:43 2026 +0530

    Initial: added mean calculation algorithm
```

Phase 6: Advanced Git Diagnostics & Workspace Management

```
PS C:\Users\tanma\cloned-project> git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Tanmay Ahuja
user.email=tanmayahuja@hotmail.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=C:/Users/tanma/./remote-mock.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
branch.master.remote=origin
branch.master.merge=refs/heads/master
PS C:\Users\tanma\cloned-project> git log -p -n 2
commit 9ea0c56490975e7a50f0dd99a50ff2edc739a3b2 (HEAD -> master, origin/master, origin/HEAD)
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:19:03 2026 +0530

    Revert "Feat: added median functionality to analytics"

    This reverts commit 1bfab817a8feaf153ea1203122e681846719b854.

diff --git a/analytics.py b/analytics.py
index d8868bc..0046447 100644
--- a/analytics.py
+++ b/analytics.py
```

```
@@ -2,11 +2,5 @@
     if not data: return 0
     return sum(data) / len(data)

-def calculate_median(data):
-    if not data: return 0
-    s = sorted(data)
-    return s[len(s)//2]
-
 numbers = [10, 20, 30, 40, 50]
 print(f"Mean: {calculate_mean(numbers)}")
-print(f"Median: {calculate_median(numbers)}")

commit 1bfab817a8feaf153ea1203122e681846719b854
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:18:45 2026 +0530

    Feat: added median functionality to analytics

diff --git a/analytics.py b/analytics.py
index 0046447..d8868bc 100644
--- a/analytics.py
+++ b/analytics.py
@@ -2,5 +2,11 @@
     if not data: return 0
     return sum(data) / len(data)

:
commit 9ea0c56490975e7a50f0dd99a50ff2edc739a3b2 (HEAD -> master, origin/master, origin/HEAD)
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:19:03 2026 +0530

    Revert "Feat: added median functionality to analytics"

    This reverts commit 1bfab817a8feaf153ea1203122e681846719b854.

diff --git a/analytics.py b/analytics.py
index d8868bc..0046447 100644
--- a/analytics.py
+++ b/analytics.py
@@ -2,11 +2,5 @@
```

```
@@ -2,11 +2,5 @@
     if not data: return 0
     return sum(data) / len(data)

-def calculate_median(data):
-    if not data: return 0
-    s = sorted(data)
-    return s[len(s)//2]
-
 numbers = [10, 20, 30, 40, 50]
 print(f"Mean: {calculate_mean(numbers)}")
-print(f"Median: {calculate_median(numbers)}")

commit 1bfab817a8feaf153ea1203122e681846719b854
Author: Tanmay Ahuja <tanmayahuja@hotmail.com>
Date:   Tue Jul 14 22:18:45 2026 +0530

    Feat: added median functionality to analytics

diff --git a/analytics.py b/analytics.py
index 0046447..d8868bc 100644
--- a/analytics.py
+++ b/analytics.py
@@ -2,5 +2,11 @@
     if not data: return 0
     return sum(data) / len(data)

+def calculate_median(data):
PS C:\Users\tanma\cloned-project> git branch
* master
PS C:\Users\tanma\cloned-project> git branch feature-cleanup
PS C:\Users\tanma\cloned-project> git branch -d feature-cleanup
Deleted branch feature-cleanup (was 9ea0c56).
PS C:\Users\tanma\cloned-project> Add-Content -Path analytics.py -Value "# DevOps Verification Complete"
PS C:\Users\tanma\cloned-project> git diff
diff --git a/analytics.py b/analytics.py
index 0046447..d4ad67a 100644
--- a/analytics.py
+++ b/analytics.py
@@ -4,3 +4,4 @@
```

```
@@ -4,3 +4,4 @@

 numbers = [10, 20, 30, 40, 50]
 print(f"Mean: {calculate_mean(numbers)}")
+# DevOps Verification Complete
PS C:\Users\tanma\cloned-project> git stash
Saved working directory and index state WIP on master: 9ea0c56 Revert "Feat: added median functionality to analytics"
PS C:\Users\tanma\cloned-project> git stash pop
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   analytics.py

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (ec0bf62384b6b72fa641965d5d01e4585d9b4f83)
PS C:\Users\tanma\cloned-project>
```

Conclusion :

In conclusion, this experiment successfully demonstrated the practical application of version control operations using the Git distributed framework. By establishing localized tracking loops, executing branch isolation workflows, and deploying error recovery mechanisms, the core competencies of source code management outlined in Learning Outcome 2 (LO2) were achieved. Furthermore, simulating remote pipeline workflows underscored the importance of configuration consistency and code traceability within collaborative Agile and DevOps engineering environments.

LO2 mapped successfully.